Listen

Catching Memory Bugs with AddressSanitizer

How to see the memory errors your program is hiding — before I do

Listen
Dictionary
Translate

CSC 240 · C++ Data Structures · Tool setup for Units 2 and beyond

> **Why this exists.** Starting in Unit 2 you manage memory by hand — `new`, `delete`, raw pointers, arrays you size yourself. When you get that wrong, your program very often *still prints the right answer* and looks completely fine. The bug is real anyway, and the grading tools compile your code with a checker called **AddressSanitizer (ASan)** that finds these silent errors. This page shows you how to run that same check yourself, so nothing catches you by surprise on a grade.

On this page

1. What ASan actually does
2. Which track are you? (read this first)
3. Windows setup — WSL
4. Mac setup — native
5. Using it: build and run
6. Reading an ASan report
7. Two worked examples
8. When something goes wrong

1. What ASan actually does

AddressSanitizer is not a separate program you install — it is a *flag* you add when you compile: `-fsanitize=address`. With it turned on, your program watches every memory access as it runs. The instant it reads or writes memory it should not — one slot past the end of an array, memory you already `delete`d, memory that was never allocated — it stops and prints a detailed report saying exactly what happened and on which line.

> **The key idea:** matching output tells you your *logic* is probably right. It tells you nothing about whether your *memory* is right. Those are two separate things, and ASan is how you check the second one. A program can print every expected line perfectly and still be corrupting memory on the way.

2. Which track are you? (read this first)

Setup depends entirely on your operating system, and the two paths are genuinely different. Find yourself here before going further:

- **Windows** — you need to do your compiling inside **WSL** (Windows Subsystem for Linux). Go to Section 3.
- **Mac** — ASan works with the compiler you already have. Go to Section 4.

> **Windows students — the one thing that will waste your afternoon:** the plain Windows C++ compilers (MinGW, Cygwin g++) **do not support AddressSanitizer.** If you add `-fsanitize=address` to one of those, it will either refuse to link or build something that crashes instantly for the wrong reason. This is not something you can fix with a setting — it is a missing feature in those compilers. The answer is WSL, below, which gives you a real Linux compiler that fully supports ASan. Do not skip it and try to make native Windows work.

3. Windows setup — WSL

WINDOWS ONLY

WSL runs a real Ubuntu Linux inside Windows. You install it once, then do your C++ work "inside" it through VS Code. It sounds heavier than it is — most of the time is just waiting on downloads.

Step 1 — Install WSL

Open **PowerShell as Administrator** (right-click the Start button → "Terminal (Admin)" or "Windows PowerShell (Admin)"), and run:

```
wsl --install
```

This installs WSL and Ubuntu. **Restart your computer when it asks.** After the restart, an Ubuntu window opens on its own and asks you to create a username and password — pick something simple and *remember the password* (you will type it for installs; it will not show characters as you type — that is normal).

> Already have WSL from before? Check with `wsl --list --verbose` in PowerShell. If you see an Ubuntu entry, you can skip installing and just open it.

Step 2 — Install the compiler inside Ubuntu

In the Ubuntu window (the Linux one, not PowerShell), run:

```
sudo apt update
sudo apt install -y g++ gdb
```

Enter your Ubuntu password when asked. This gives you the Linux `g++` compiler, which supports ASan fully.

Step 3 — Get the VS Code WSL extension

In VS Code: open Extensions (`Ctrl+Shift+X`), search **WSL**, and install the one published by Microsoft.

Step 4 — Open your lab folder *inside* WSL

This is the step everyone gets wrong once. Your files must be opened *through* WSL so the Linux compiler is used. The reliable way: in the Ubuntu window, move into your project folder and launch VS Code from there.

```
# make a folder in your Linux home and go into it
mkdir -p ~/csc240/lab && cd ~/csc240/lab

# launch VS Code, attached to WSL
code .
```

The first `code .` installs a small helper into WSL — give it a few seconds.

> **Checkpoint — do not continue until this is true:** the **bottom-left corner** of the VS Code window shows a green box that says **WSL: Ubuntu**. If it says anything else, or there is no green box, you are still in plain Windows and ASan will not work. Re-do `code .` from the Ubuntu window.

> **Keep your code on the Linux side.** Put your `.cpp` and `.h` files under your Ubuntu home (paths starting with `~/` or `/home/`), not on the Windows side (`/mnt/c/...`). It runs faster and ASan behaves better. To bring a file in from Windows, copy it: your Windows Desktop is reachable at `/mnt/c/Users/YOURNAME/Desktop/` .

4. Mac setup — native

MAC ONLY

Good news: the compiler that ships with macOS supports AddressSanitizer out of the box. There is almost nothing to install.

Step 1 — Make sure the command-line tools are installed

Open **Terminal** (Applications → Utilities → Terminal) and run:

```
clang++ --version
```

If it prints version information, you are ready — skip to Step 3. If macOS pops up a box offering to install "command line developer tools," accept it and wait for it to finish.

Step 2 — (If the pop-up did not appear) install the tools

```
xcode-select --install
```

Follow the prompt, then re-run `clang++ --version` to confirm.

Step 3 — Get the VS Code C/C++ extension

In VS Code: Extensions (`Cmd+Shift+X`), search **C/C++**, install the one published by Microsoft. Then just open your lab folder normally (File → Open Folder).

> On Mac, `g++` is usually the same Apple compiler as `clang++` under a different name, and both accept `-fsanitize=address` . The commands in the rest of this page work as written.

5. Using it: build and run

You can run ASan two ways. The terminal command is the one to learn first — it always works and it is exactly what the grading tool does. The one-key VS Code button is a convenience on top.

The command (learn this one)

Open the terminal inside VS Code (`Ctrl+`` — the backtick key, top-left of your keyboard), make sure you are in your project folder, and compile with the sanitizer on:

```
g++ -std=c++11 -Wall -Wuninitialized -g -fsanitize=address YOURFILE.cpp -o check
```

Then run what you just built:

```
./check
```

Replace `YOURFILE.cpp` with the file that has `int main()`. These are the same flags the grader uses, so what you see here is what it sees. If your program has no memory errors, it runs normally and exits with no extra output. If it does, you get a report — see the next section.

> **Run the file you just built.** If you fix a bug, you must compile *again* before running, or you are just re-running the old broken program and the error will look like it never went away. Recompile, then `./check`.

The one-key button (optional convenience)

To build with `Ctrl+Shift+B` instead of typing the command each time, create a file called `.vscode/tasks.json` in your project folder with exactly this content:

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "type": "cppbuild",
            "label": "Build with AddressSanitizer",
            "command": "g++",
            "args": [
                "-std=c++11", "-Wall", "-Wuninitialized", "-g",
                "-fsanitize=address",
                "${file}",
                "-o", "${fileDirname}/${fileBasenameNoExtension}"
            ],
            "options": { "cwd": "${fileDirname}" },
            "problemMatcher": ["$gcc"],
            "group": { "kind": "build", "isDefault": true }
        }
    ]
}
```

Now, with a `.cpp` file open, `Ctrl+Shift+B` compiles it with ASan on. It builds but does not run — run the result from the terminal as above (the program is named after your file, with no extension).

> **Note on multi-file labs.** This button compiles only the one file you have open. That is fine when your whole implementation lives in a header (as in Lab 5) or in a single `.cpp`. When a lab splits its implementation across several `.cpp` files that must be compiled together, use the terminal command instead and list them all: `g++ ... driver.cpp thing.cpp other.cpp -o check`.

6. Reading an ASan report

An ASan report looks alarming — lots of red, hex addresses, a "shadow bytes" block at the bottom. You can ignore almost all of it. Three lines carry the meaning:

- The **first line** names the error type — e.g. `heap-buffer-overflow` (touched memory outside an array), `heap-use-after-free` (used memory you already deleted), `double-free` (deleted the same thing twice).
- The **SUMMARY line** (near the bottom) gives you the *file and line number* where it happened, and the function name. This is where you start looking.
- The **"allocated by" / "freed by"** blocks tell you where that memory was originally created and, if relevant, where it was freed — which is often a different line than the crash, and is usually the real cause.

Everything else — the shadow bytes grid, the raw addresses — is for advanced debugging and you do not need it for this course. Read the first line and the SUMMARY line, go to that line in your code, and think about what memory it touches.

> **Example.** `SUMMARY: AddressSanitizer: heap-buffer-overflow sorted.h:303 in DeleteItem` means: line 303 of `sorted.h`, inside `DeleteItem`, read or wrote one step outside an array. Go look at what that line indexes into, and ask whether the index can ever be one too big.

7. Two worked examples

These are the two memory bugs you are most likely to write in this course. For each one, here is a tiny program that has the bug, the report ASan actually prints when you run it, and how to fix it. Compile each yourself with the command from Section 5 and watch the report appear — then watch it disappear once fixed. That loop is the whole skill.

Example A — use-after-free (using memory you already deleted)

A classic version: you release an array, then read from it anyway. On your machine it might even print `90` and look fine — the memory has not been handed to anything else *yet*. That is luck, not correctness, and ASan calls it out.

```
#include <iostream>
using namespace std;

int main() {
    int* scores = new int[3];
    scores[0] = 90; scores[1] = 85; scores[2] = 78;
```

```
    delete[] scores;              // memory released here

    cout << scores[0] << endl;    // ...but read here, AFTER the free
    return 0;
}
```

Running it produces (trimmed to the lines that matter):

```
==ERROR: AddressSanitizer: heap-use-after-free on address 0x...
READ of size 4 at 0x... thread T0
    #0 ... in main uaf.cpp:10          <- where you USED it

freed by thread T0 here:
    #1 ... in main uaf.cpp:8           <- where you delete[]d it

previously allocated by thread T0 here:
    #1 ... in main uaf.cpp:5           <- where you new[]d it

SUMMARY: AddressSanitizer: heap-use-after-free uaf.cpp:10 in main
```

Notice how much the report hands you: line 10 is where you touched freed memory, line 8 is where you freed it, line 5 is where it was born. The bug is the mismatch between line 8 and line 10 — you used the pointer after releasing what it points to.

> **The fix:** do not read through a pointer after `delete`ing what it points to. Either move the read *before* the `delete[]`, or do not delete until you are truly done with the data. In your real labs this shows up in subtler forms — e.g. a function that takes a list *by value*, so a copy's destructor frees nodes the original still points at. Same bug, same report.

Example B — missing destructor (a memory leak)

Starting in Unit 3, a class that allocates memory needs a destructor to release it. Leave the destructor out and every object of that class leaks whatever it allocated. Nothing crashes; the program prints `done` and exits — and ASan reports the leak at exit.

```
#include <iostream>
using namespace std;

class IntList {
public:
    IntList(int n) { data = new int[n]; }   // allocates in the constructor
    // NO destructor -- the new[] is never matched with a delete[]
private:
    int* data;
};
```

```
int main() {
    IntList list(100);   // 100 ints allocated; when 'list' dies, nothing frees them
    cout << "done" << endl;
    return 0;
}
```

The program runs fine, then at exit you get:

```
done
==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 400 byte(s) in 1 object(s) allocated from:
    #1 ... in IntList::IntList(int) leak.cpp:6      <- the new[] that leaked
    #2 ... in main leak.cpp:13                       <- where the object was created

SUMMARY: AddressSanitizer: 400 byte(s) leaked in 1 allocation(s).
```

Two things to notice. First, the first line says **LeakSanitizer**, not AddressSanitizer — that is the leak-detecting part of ASan; it is the same tool, and a leak is a real defect, not a warning to ignore. Second, the report points at the *allocation* (line 6, the `new[]` in the constructor), because that is the memory that was never released — it cannot point at the missing `delete[]`, since there isn't one. The fix is to add the thing that isn't there.

The fix: give the class a destructor that releases what the constructor allocated:

```
~IntList() { delete[] data; }
```

Add that, recompile, run again — the leak report is gone. And once a class needs a destructor, ask the Unit 3 question: does it also need a copy constructor and an assignment operator? (See the Rule of Three note.)

8. When something goes wrong

What you see	What it means / what to do
`cannot find -lasan` or undefined references to `__asan_*` when linking (Windows)	You are compiling with a native-Windows compiler that lacks ASan. You are not inside WSL — check for the green **WSL: Ubuntu** badge (Section 3, Step 4).
`g++: command not found` (Windows/WSL)	The compiler is not installed inside Ubuntu yet. Run the `apt install` from Section 3, Step 2.

What you see	What it means / what to do
The bottom-left corner does not say **WSL: Ubuntu**	You opened the folder as a normal Windows folder. Close it, and from the Ubuntu window `cd` to your project and run `code .` again.
You fixed the bug but the same error still appears	You ran the old binary. Recompile first, *then* run again. (Very common — build and run are two separate steps.)
`clang++: command not found` (Mac)	Command-line tools are not installed. Run `xcode-select --install` (Section 4, Step 2).
Report says a line in a system file, not your code	Read further down to the first line that names *your* file — that is the one that matters. The SUMMARY line usually points straight to it.

> **Still stuck?** Bring it to office hours — virtual, 30-minute slots, Monday through Friday. A setup problem is a five-minute fix in person and worth not fighting alone. Paste the exact command you ran and the full output when you ask; that is usually all I need to see what is happening.
